MINI PROJECT REPORT

Shortest Path Finder – Road Network using Graph (C++ / No STL)

Project Overview

This project represents locations as vertices and two-way roads as weighted edges. An adjacency matrix stores road distances in kilometres. The program supports adding locations and roads, displaying the network, searching locations, BFS, DFS, and finding the shortest route using Dijkstra's algorithm.

Source Code

```
// Mini Project: Shortest Path Finder - Road Network using Graph (no STL)
// Graph is WEIGHTED and UNDIRECTED (every road is two-way).
// Stored as an adjacency matrix: road[a][b] = distance in km, 0 = no road.
#include <iostream>
#include <cstring>
#include <cstdlib>
using namespace std;

const int MAX = 10;     // maximum number of locations
const int INF = 99999;  // "infinity" for Dijkstra (bigger than any route: 9 roads x 10000 km)

class Graph {
    char name[MAX][20];   // name of each location
    int road[MAX][MAX];   // adjacency matrix of distances
    int n;                // number of locations added so far

    // DFS helper (recursion): visit v, then each unvisited neighbour
    void dfsVisit(int v, bool seen[]) {
        seen[v] = true;
        cout << name[v] << " ";
        for (int i = 0; i < n; i++)
            if (road[v][i] != 0 && !seen[i]) dfsVisit(i, seen);
    }

public:
    Graph() {                                 // constructor: empty network
        n = 0;
        for (int i = 0; i < MAX; i++)
            for (int j = 0; j < MAX; j++) road[i][j] = 0;
    }

    int find(const char* s) {                 // LINEAR SEARCH, -1 if absent
        for (int i = 0; i < n; i++)
            if (strcmp(name[i], s) == 0) return i;
        return -1;
    }

    void addLocation(const char* s) {
        if (strlen(s) == 0) cout << "Invalid name.\\n";
        else if (n == MAX) cout << "Network full (max " << MAX << ").\\n";
        else if (find(s) != -1) cout << "Location already exists.\\n";
        else { strcpy(name[n++], s); cout << s << " added.\\n"; }
    }

    void addRoad(const char* a, const char* b, int d) {
        int x = find(a), y = find(b);
        if (x == -1 || y == -1) cout << "Location not found.\\n";
        else if (x == y) cout << "A road cannot join a place to itself.\\n";
        else if (d <= 0 || d > 10000) cout << "Distance must be 1 to 10000 km.\\n";
```

```
        else { road[x][y] = road[y][x] = d; cout << "Road saved.\\n"; }
    }

    void display() {
        if (n == 0) { cout << "Network is empty.\\n"; return; }
        for (int i = 0; i < n; i++) {
            cout << name[i] << " -> ";
            for (int j = 0; j < n; j++)
                if (road[i][j] != 0) cout << name[j] << "(" << road[i][j] << "km) ";
            cout << "\\n";
        }
    }

    void bfs(const char* start) {              // BFS with array-based queue
        int s = find(start);
        if (s == -1) { cout << "Location not found.\\n"; return; }
        int line[MAX], front = 0, rear = 0;   // array used as a queue
        bool seen[MAX] = {false};
        line[rear++] = s; seen[s] = true;
        while (front < rear) {
            int v = line[front++];            // dequeue
            cout << name[v] << " ";
            for (int i = 0; i < n; i++)
                if (road[v][i] != 0 && !seen[i]) { seen[i] = true; line[rear++] = i; }
        }
        cout << "\\n";
    }

    void dfs(const char* start) {
        int s = find(start);
        if (s == -1) { cout << "Location not found.\\n"; return; }
        bool seen[MAX] = {false};
        dfsVisit(s, seen);
        cout << "\\n";
    }

    void shortest(const char* from, const char* to) {   // DIJKSTRA
        int s = find(from), t = find(to);
        if (s == -1 || t == -1) { cout << "Location not found.\\n"; return; }
        int dist[MAX], prev[MAX];
        bool done[MAX] = {false};
        for (int i = 0; i < n; i++) { dist[i] = INF; prev[i] = -1; }
        dist[s] = 0;
        for (int round = 0; round < n; round++) {
            int u = -1;                       // unfinished vertex with least distance
            for (int i = 0; i < n; i++)
                if (!done[i] && dist[i] < INF && (u == -1 || dist[i] < dist[u])) u = i;
            if (u == -1) break;               // rest are unreachable
            done[u] = true;
            for (int v = 0; v < n; v++)       // relax roads leaving u
                if (road[u][v] != 0 && dist[u] + road[u][v] < dist[v]) {
                    dist[v] = dist[u] + road[u][v];
                    prev[v] = u;
                }
        }
        if (dist[t] == INF) { cout << "No route between " << from << " and " << to << ".\\n"; return; }
        int path[MAX], len = 0;               // walk back from t using prev[]
        for (int v = t; v != -1; v = prev[v]) path[len++] = v;
        cout << "Shortest distance: " << dist[t] << " km\\nRoute: ";
        for (int i = len - 1; i >= 0; i--) cout << name[path[i]] << (i > 0 ? " -> " : "\\n");
    }
};

// Read one line of text; text becomes "" if the line is too long.
void readLine(const char* prompt, char* text) {
    cout << prompt;
    if (!cin.getline(text, 20)) {
        if (cin.eof()) exit(0);
        cin.clear(); cin.ignore(1000, '\\n'); text[0] = '\\0';
```

```
    }
}

int main() {
    Graph g;
    char a[20], b[20], num[20];
    int choice;
    do {
        cout << "\\n1.Add location  2.Add road  3.Display  4.Search\\n"
             << "5.Shortest route  6.BFS  7.DFS  8.Exit\\n";
        readLine("Choice: ", num);
        choice = atoi(num);                    // "abc" gives 0 = invalid
        if (choice == 1) { readLine("Name: ", a); g.addLocation(a); }
        else if (choice == 2) {
            readLine("From: ", a); readLine("To: ", b); readLine("Distance (km): ", num);
            g.addRoad(a, b, atoi(num));
        }
        else if (choice == 3) g.display();
        else if (choice == 4) {
            readLine("Search: ", a);
            int i = g.find(a);
            if (i == -1) cout << "Not found.\\n"; else cout << "Found at index " << i << ".\\n";
        }
        else if (choice == 5) { readLine("From: ", a); readLine("To: ", b); g.shortest(a, b); }
        else if (choice == 6) { readLine("Start: ", a); g.bfs(a); }
        else if (choice == 7) { readLine("Start: ", a); g.dfs(a); }
        else if (choice != 8) cout << "Invalid choice.\\n";
    } while (choice != 8);
    return 0;
}
```

Sample Program Output

The following is a cleaned presentation of the sample run supplied with the project. The initial linker message is retained below as a note.

```
Initial compile attempt:
g++ main.cpp -o main
Linker error: cannot open output file main.exe: Permission denied
This commonly occurs when main.exe is still running or locked. Close the running program/terminal process and compile again, or use another output filename.

PS C:\\Users\\Admin\\OneDrive\\Desktop\\dsa\\project2cce3> g++ main.cpp -o main
c:/mingw/bin/../lib/gcc/mingw32/6.3.0/../../../../mingw32/bin/ld.exe: cannot open output file main.exe: Permission denied
collect2.exe: error: ld returned 1 exit status
PS C:\\Users\\Admin\\OneDrive\\Desktop\\dsa\\project2cce3> ./main

1.Add location  2.Add road  3.Display  4.Search
5.Shortest route  6.BFS  7.DFS  8.Exit
Choice: 1
Name: Pune
Pune added.

1.Add location  2.Add road  3.Display  4.Search
5.Shortest route  6.BFS  7.DFS  8.Exit
Choice: 1
Name: Mumbai
Mumbai added.

1.Add location  2.Add road  3.Display  4.Search
5.Shortest route  6.BFS  7.DFS  8.Exit
Choice: 1
Name: Nashik
Nashik added.

1.Add location  2.Add road  3.Display  4.Search
5.Shortest route  6.BFS  7.DFS  8.Exit
Choice: 1
Name: Nagpur
Nagpur added.

1.Add location  2.Add road  3.Display  4.Search
5.Shortest route  5.BFS  6.DFS  8.Exit
Choice: 2
From: Pune
To: Mumbai
Distance (km): 150
Road saved.

Choice: 2
From: Pune
To: Nashik
Distance (km): 210
Road saved.

Choice: 2
From: Mumbai
To: Nashik
Distance (km): 180
Road saved.

Choice: 2
From: Nashik
To: Nagpur
Distance (km): 300
Road saved.

Choice: 3
Pune -> Mumbai(150km) Nashik(210km)
Mumbai -> Pune(150km) Nashik(180km)
```

```
Nashik -> Pune(210km) Mumbai(180km) Nagpur(300km)
Nagpur -> Nashik(300km)

Choice: 4
Search: Pune
Found at index 0.

Choice: 4
Search: Nagpur
Found at index 3.

Choice: 5
From: Nashik
To: Nagpur
Shortest distance: 300 km
Route: Nashik -> Nagpur

Choice: 2
From: Pune
To: Nashik
Distance (km): 400
Road saved.

Choice: 5
From: Pune
To: Nashik
Shortest distance: 330 km
Route: Pune -> Mumbai -> Nashik

Choice: 6
Start: Pune
Pune Mumbai Nashik Nagpur

Choice: 7
Start: Mumbai
Mumbai Pune Nashik Nagpur

Choice: 1
Name: Pune
Location already exists.

Choice: 1
Name:
Invalid name.

Choice: 1
Name: Mumbai
Location already exists.

Choice: 6
Start: Delhi
Location not found.

Choice: 7
Start: Delhi
Location not found.

Choice: 5
From: Pune
To: Goa
Location not found.

Choice: 8
Exit
```

Key Result

For the updated road distances, the shortest route from Pune to Nashik is **Pune** → **Mumbai** → **Nashik**, with a total distance of **330 km** (150 km + 180 km). BFS and DFS both visit all four connected locations in this example; their traversal order depends on the order in which neighbours are checked in the matrix.